Lab 4: Automated Deployment (CD)

I. Learning Objectives

After completing this lab, you will be able to:

- Implement a basic Continuous Deployment (CD) pipeline.
- Configure GitHub Actions to **automatically deploy** your app after a merge to `main`.
- Securely manage server credentials (SSH key, IP, user) using **GitHub Secrets**.
- Use an SSH action to remotely run `docker-compose` on a production server.

II. Prerequisites

- **Lab 3 Completed:** Your project's `ci.yml` file must successfully run tests and push an image to Docker Hub.
- **Server Credentials:** Your lecturer will provide you with:
  1. Server IP Address (e.g., `123.45.67.89`)
  2. Server Username (e.g., `root` or `ubuntu`)
  3. An SSH Private Key file (e.g., `id_rsa.pem`)
- **New Branch:** All work for this lab MUST be done on a new feature branch.

```
# Make sure you're on main and have the latest changes
$ git checkout main
$ git pull origin main

# Create a new branch for this lab's work
$ git checkout -b feature/add-cd-pipeline
```

III. Part A: Server-Side Setup

First, we must prepare your "production" server to receive the application.

Step A.1: Test Your SSH Connection

Before automating anything, ensure you can log in to the server manually from your local machine.

1. Save the SSH key file (`.pem`) provided by your lecturer to a secure location (e.g., `~/.ssh/`).
2. Set the correct permissions for the key (this is a one-time command):

```
# This makes the key file read-only for you
$ chmod 400 /path/to/your/id_rsa.pem
```

3. Connect to the server:

```
$ ssh -i /path/to/your/id_rsa.pem username@server-ip-address
```

If you see a welcome message (e.g., "*Welcome to Ubuntu...*"), you are in! Type `exit` to log out. If not, troubleshoot with your lecturer.

Step A.2: Install Software on the Server

Log back into the server with the `ssh` command. You need to install `git`, `docker`, and `docker-compose`.

> **Note:** `docker` and `docker-compose` may already be installed for you. You can check with `docker --version` and `docker-compose --version`.

```
# --- RUN THESE COMMANDS ON THE SERVER ---


# 1. Update package lists
$ sudo apt-get update
```

```
# 2. Install Git
$ sudo apt-get install -y git

# 3. (If needed) Install Docker & Docker Compose
$ sudo apt-get install -y docker.io
$ sudo apt-get install -y docker-compose
```

Step A.3: Clone Your Project (On the Server)

Your server needs a copy of the project, especially the `docker-compose.yml` file.

```
# --- RUN THESE COMMANDS ON THE SERVER ---
# 1. Create a directory for your app (optional, but good practice)
$ sudo mkdir -p /var/www
# 2. Go into that directory
$ cd /var/www

# 3. Clone your group's repository (use the HTTPS URL)
$ sudo git clone https://github.com/Team-
Leader/devops-group-1.git

# 4. Go into your project folder
$ cd devops-group-1

# 5. VERY IMPORTANT: Note this path! You will need it.
# The path is: /var/www/devops-group-1
```

Log out of the server by typing `exit`. The rest of the work is on your local machine and GitHub.

IV. Part B: Configure GitHub Secrets

We need to give GitHub Actions the "keys" to our server, but we must do it securely.

> **CRITICAL:** NEVER paste your private key or server IP directly into the `ci.yml` file. This is a massive security breach.

Step B.1: Add Server Secrets

Go to your **group's GitHub repo** > `Settings` > `Secrets and variables` > `Actions`.

You already have `DOCKER_HUB_USERNAME` and `DOCKER_HUB_PASSWORD`. Now add these three:

1. **Secret 1: `HOST`**
   - Name: `HOST`
   - Value: `(Your server's IP address, e.g., 123.45.67.89)`
2. **Secret 2: `USERNAME`**
   - Name: `USERNAME`
   - Value: `(Your server's username, e.g., root)`
3. **Secret 3: `SSH_PRIVATE_KEY`**
   - Name: `SSH_PRIVATE_KEY`
   - Value: (Open your `.pem` file in a text editor. Copy **everything**, from `-----BEGIN RSA PRIVATE KEY-----` all the way to `-----END RSA PRIVATE KEY-----`, and paste it into the value box.)

You should now have 5 secrets in total.

V. Part C: Add the "Deploy" Job to `ci.yml`

This is the final piece. We will add a second job to our workflow that runs **after** the build-and-test job.

Step C.1: Edit `ci.yml` Locally

On your local machine, on the `feature/add-cd-pipeline` branch, open `.github/workflows/ci.yml`.

Step C.2: Define the `deploy` Job

At the **end** of the file, at the same indentation level as `build-and-test`, add the new `deploy` job.

```
jobs:
  build-and-test:
# ... (Your entire job from Lab 3 is here) ...
# ... (It ends with the 'docker/build-push-action' step) ...

  deploy: # <-- This is our new job
    name: Deploy to Server
    runs-on: ubuntu-latest

    # This is critical!
    needs: build-and-test # 1. Wait for build-and-test to
succeed

    if: github.ref == 'refs/heads/main' # 2. *Only* run
this job on a push to 'main'
```

> **Why these two conditions?**
> 1. `needs: build-and-test`: We don't want to deploy if the tests failed or the image build failed.
> 2. `if: github.ref == ...`: We don't want to deploy every single Pull Request. We **only** want to deploy when a PR is merged into `main`.

Step C.3: Add the SSH Deployment Step

Now, add the `steps` to your new `deploy` job. We will use a pre-built Action to handle the SSH connection.

```yaml
# ... (at the end of your 'deploy:' job definition) ...
steps:
  - name: Deploy to Server via SSH
    uses: appleboy/ssh-action@master
    with:
      host: ${{ secrets.HOST }}
      username: ${{ secrets.USERNAME }}
      key: ${{ secrets.SSH_PRIVATE_KEY }}
      script: |
          # 1. Go to the project directory (THIS PATH IS FROM
STEP A.3)
           cd /var/www/devops-group-1

          # 2. Pull the latest code from the 'main' branch sudo
git pull origin main

          # 3. Pull the new Docker image(s) from Docker Hub sudo
docker-compose pull

          # 4. Stop, remove, and restart the containers with the
new image
          sudo docker-compose up -d
```

Critical:
1. Update the `cd /var/www/devops-group-1` path to match the exact path you used in Step A.3.
2. We use `sudo` because Docker and Git on the server are likely installed for the root user.

VI. Part D: The Final Test

Step D.1: Commit, Push, and Open PR

```
# 1. Add the updated CI file
$ git add .github/workflows/ci.yml


# 2. Commit it
$ git commit -m "ci: Add automated CD deploy job"
```

```
# 3. Push your branch
$ git push origin feature/add-cd-pipeline
```

Go to GitHub and open a Pull Request for your branch.

Step D.2: Observe the "Skipped" Job

On your new PR, go to the "Checks" tab. You will see your workflow run.

- The `Build & Test` job will run (and pass).
- The `Deploy to Server` job will be **SKIPPED**.

This is **correct!** It skipped because our `if:` condition (`github.ref == 'refs/heads/main'`) is false. We are on a feature branch, not `main`.

Step D.3: MERGE THE PULL REQUEST

Get your teammates to review and approve the PR. Then, the Team Leader must **Merge** it into the `main` branch.

Step D.4: Watch the Deployment

The moment you merge, a `push` event is fired on the `main` branch. This triggers a **new workflow run**.

> **Task:** Go to your repository's "Actions" tab. Click on the new workflow that just started (it will say it's running on the `main` branch).
>
> 1. Watch the `Build & Test` job run and pass.
> 2. When it finishes, watch the `Deploy to Server` job start (it's no longer skipped!).
> 3. Click the `Deploy to Server` job and watch its logs. You should see it connect via SSH and successfully run your `git pull` and `docker-compose` commands.

Step D.5: VERIFY YOUR DEPLOYMENT

If the `Deploy to Server` job gets a green checkmark, open your browser.

Go to your server's public IP address, using the port you mapped in `docker-compose.yml` (e.g., `http://123.45.67.89:8080`).

You should see your application running, live on the internet, deployed 100% automatically.

Step D.6: (Final Proof) Deploy a Change

To prove it works end-to-end, let's deploy a small visual change.

1. On your local machine, create a new branch (e.g., `feature/homepage-text`).
2. Change some text in your `src/` files (e.g., change the homepage title).
3. Commit, push, and open a PR.
4. Watch the CI checks pass (deploy is skipped).
5. **Merge the PR.**
6. Go to the "Actions" tab, watch the new `main` workflow run.
7. When the `deploy` job finishes, **refresh your browser** on the server's IP.

Your text change should appear instantly. You have now built a full, automated CI/CD pipeline.